CLAUDE CERTIFIED ARCHITECT — PROFESSIONAL

Objective 13: Analyse Authentication and Authorisation

Exam objective: Analyse authentication and authorisation requirements for Claude-based integrations, including the distinction between authentication and authorisation, OAuth 2.1 fundamentals, API keys and credentials, delegated user identity, token handling and audience validation, least-privilege permissions, role-based access control, tenant isolation, secure secrets management, and preventing privilege escalation.

Q1 **A healthcare platform's Claude-based assistant calls an internal FHIR API using a single shared service-account API key for all users. Security review flags this as a problem. What is the core issue?**

A **The API key is authentication without per-user authorisation, so the system cannot enforce which patient records an individual user should be allowed to see**

B API keys are inherently insecure and should never be used

C The FHIR API does not support API keys

D Shared keys always expire too quickly to be practical

CORRECT ANSWER **A**

WHY A shared service key authenticates the application to the API but carries no user-level identity, so authorisation decisions (who can see which records) cannot be enforced downstream — this conflates authentication with authorisation. API keys themselves are a valid credential type in many contexts; the problem here is architectural, not that keys are inherently unsafe or short-lived.

Q2 **An architect is designing OAuth 2.1 flows for a Claude agent that needs to act on behalf of a logged-in enterprise user against a third-party calendar API. Which grant type is most appropriate?**

A Resource Owner Password Credentials grant, so the agent can store the user's password directly

B **Authorization Code flow with PKCE, delegating user consent and issuing a scoped access token to the agent**

C Implicit grant, since it is the newest and simplest flow in OAuth 2.1

D Client Credentials grant using the agent's own identity only

CORRECT ANSWER **B**

WHY OAuth 2.1 mandates Authorization Code flow with PKCE as the recommended flow for delegated user authorization, ensuring the user consents and the agent receives a scoped token representing the user, not the agent itself. The Implicit grant is removed in OAuth 2.1, Resource Owner Password Credentials requires handling raw user passwords (a serious anti-pattern), and Client Credentials represents the app's own identity, not the user's.

Q3 **A financial services firm issues a Claude agent a long-lived, unscoped bearer token that grants full read/write access to the core banking API, used across all agent functions. What principle is being violated?**

A Tenant isolation

B **Least privilege**

C Token audience validation

D Progressive discovery

CORRECT ANSWER **B**

WHY Issuing a single broad, long-lived token for all functions violates least privilege, which requires granting only the minimum scope and duration needed for each specific task. This is not a tenant-isolation issue (no cross-customer boundary is described), not strictly an audience-validation issue (the token's intended recipient isn't the stated concern), and unrelated to progressive tool discovery.

Q4 A multi-tenant SaaS company's Claude agent processes requests for Customer A and Customer B using the same backend. A bug causes a token issued for Customer A's session to be reused when answering Customer B's request. What security property has failed?

A Rate limiting

B Tenant isolation

C Prompt caching

D Reranking accuracy

CORRECT ANSWER B

WHY Tenant isolation ensures that credentials, data, and sessions for one customer cannot leak into another's context; a reused token across tenants is a direct tenant-isolation failure with potential data-breach consequences. Rate limiting, prompt caching, and reranking are unrelated controls that do not govern cross-tenant credential boundaries.

Q5 A government agency's Claude integration validates that a JWT's signature is valid and not expired, but does not check the `aud` (audience) claim before using it to call an internal records API. What risk does this introduce?

A Slower token issuance

B A token legitimately issued for a different service could be replayed against this API, since the API accepts any validly signed token regardless of intended recipient

C Increased latency in retrieval

D Reduced context window utilization

CORRECT ANSWER B

WHY Audience validation ensures a token is only accepted by the service(s) it was actually issued for; skipping this check enables token replay/confusion across services that trust the same issuer, a well-known JWT vulnerability class. This is a security correctness issue, not a performance or retrieval-quality concern.

Q6 An enterprise wants Claude's agent to perform actions "as the user" when integrating with Microsoft Graph (email, calendar). Which approach best preserves accurate authorisation boundaries?

A. **Use delegated permissions with the signed-in user's identity and consent, so Graph enforces exactly what that user is allowed to do**

B. Use application permissions with admin consent for all scopes, so the agent never has to deal with user tokens

C. Hard-code a single global admin token for all users to simplify integration

D. Bypass Graph's permission model entirely by calling a cached export of all mailboxes

CORRECT ANSWER A

WHY Delegated permissions tied to the authenticated user's identity ensure the downstream API enforces the same access boundaries the user already has, which is the correct model for "acting as the user." Application permissions with full admin consent, a global admin token, or bypassing the permission model via bulk exports all grant far broader access than necessary and violate least privilege and user-boundary expectations.

Q7 **A retail company stores Claude API keys and third-party OAuth client secrets directly in application source code committed to a private Git repository. What is the primary architectural concern?**

A Private repositories make this an acceptable long-term practice

B **Secrets in source control are a security anti-pattern regardless of repository visibility; they should be stored in a dedicated secrets manager with rotation and access auditing**

C This approach is fine as long as the secrets are base64-encoded

D This only matters if the repository is ever made public

CORRECT ANSWER **B**

WHY Secrets belong in a managed secrets store (e.g., a vault service) with rotation, access control, and audit logging; embedding them in source control creates risk through history retention, broad developer access, and lack of rotation, independent of repo visibility. Base64 encoding is not encryption and provides no real protection, and the risk exists even for private repos due to insider threat, misconfigured access, and leak-through-fork scenarios.

Q8 **A logistics company implements role-based access control (RBAC) for its Claude-powered warehouse agent: "dispatcher" role can view and reassign shipments, "viewer" role can only read shipment status. A bug causes the agent to execute reassignment tool calls for users with only the "viewer" role. What architectural layer failed?**

A Authentication, because the viewer's login credentials were invalid

B Authorisation enforcement, because the system failed to check the user's role/permissions before executing a privileged action

C Embedding quality in the retrieval layer

D Prompt caching configuration

CORRECT ANSWER **B**

WHY The user successfully authenticated (identity was established); the failure is that the system did not enforce the role-based permission check before allowing a privileged tool call, which is an authorisation failure, not an authentication one. Retrieval embeddings and prompt caching have no bearing on enforcing role-based tool permissions.

Q9 **An architect is reviewing a design where a Claude agent's tool-calling layer accepts a `user_role` parameter supplied directly by the client application in the request body, and the backend trusts it to decide what actions are permitted. What is the vulnerability?**

A. This is secure because the role is passed in a well-structured JSON field

B. This allows privilege escalation: a malicious or compromised client could simply set `user_role` to "admin" to bypass authorisation checks

C. It has no security impact because Claude itself validates the role internally

D. It only matters if TLS is not used for the request

CORRECT ANSWER **B**

WHY Authorisation decisions must be derived from a trusted, server-validated identity (e.g., a verified token's claims), never from client-supplied fields, or users can trivially escalate privileges by altering the request. Structured JSON formatting and TLS transport encryption do not address trust of the data's origin, and Claude itself has no built-in enforcement of application-level roles.

Q10 **A manufacturing company wants its Claude agent to call a partner's API using machine-to-machine authentication, with no human user involved in the transaction. Which OAuth 2.1 grant is designed for this scenario?**

A. Authorization Code flow with PKCE

B. Client Credentials grant

C. Device Authorization Grant

D. Resource Owner Password Credentials grant

CORRECT ANSWER **B**

WHY Client Credentials grant is designed for service-to-service (machine-to-machine) authentication where there is no end user to delegate on behalf of, issuing a token representing the application/service itself. Authorization Code with PKCE and Device Authorization Grant both require a human user to authenticate and consent, and Resource Owner Password Credentials is deprecated in OAuth 2.1 and requires handling user passwords directly.

Q11 **A legal services firm's Claude agent integration stores refresh tokens for connected third-party document systems in plaintext in an application database accessible to all backend engineers. What combination of controls should be prioritized?**

A. No action needed since refresh tokens cannot be used without the matching access token

B. Encrypt tokens at rest, restrict database access to least-privilege roles, and rotate/revoke tokens on a defined schedule or on suspected compromise

C. Only encrypt access tokens, since refresh tokens are short-lived by design

D. Move the tokens into client-side local storage instead

CORRECT ANSWER B

WHY Refresh tokens are long-lived and powerful (they can mint new access tokens), so they require encryption at rest, strict least-privilege database access, and rotation/revocation capability — a defense-in-depth approach to secrets management. Refresh tokens can typically be used independently to obtain new access tokens, so assuming safety is wrong; refresh tokens are usually longer-lived than access tokens, not shorter; and client-side storage is far less secure than a properly controlled backend store.

Q12 **An insurance company's Claude agent uses delegated user tokens to query a claims database, but the database connection pool used by the agent's backend service is shared and configured with a single superuser credential regardless of which end user initiated the request. What is the architectural risk?**

A. None, because the delegated token is still checked at the Claude API layer

B. The database layer itself enforces no per-user boundary, so a flaw in application-layer logic could allow any user's request to access any other user's claims data

C. This is a performance optimization with no security implications

D. This only matters for read operations, not writes

CORRECT ANSWER B

WHY If the database itself trusts a single superuser credential regardless of caller identity, authorisation is enforced only in application code; any bug, injection, or logic error in that layer removes the last line of defense, creating a serious data-exposure risk across all operations, both read and write. Claude's own token validation does not substitute for backend data-layer access controls.

Q13 **A telecom provider wants to prevent a Claude agent from silently gaining broader permissions than intended when it chains multiple tool calls (e.g., using a read-only reporting token to indirectly trigger an administrative action via a second service). What design pattern most directly mitigates this risk?**

A Granting the agent a single all-powerful token to simplify tool chaining

B **Enforcing scope and permission checks independently at each service boundary, so no downstream service implicitly trusts upstream token scope beyond what it was issued for**

C Disabling all logging to reduce the attack surface

D Allowing any token to be silently upgraded to admin scope if a tool call fails

CORRECT ANSWER B

WHY Defense-in-depth requires every service in a call chain to independently validate the token's scope and reject out-of-scope actions, preventing privilege escalation through chained calls ("confused deputy" problems). Granting one powerful token magnifies blast radius, disabling logging removes detection capability without reducing risk, and silently upgrading scope on failure is itself a dangerous escalation path.

Q14 **A government agency requires that different departments using a shared Claude deployment cannot access each other's documents, even though they share the same underlying vector database and retrieval pipeline. Which combination of controls best enforces this at the integration layer?**

A Rely solely on prompt instructions telling Claude not to retrieve cross-department documents

B **Apply metadata-based access filters tied to the authenticated user's department/tenant at query time, combined with document-level permission checks enforced server-side before results reach the model**

C Use a single shared index with no filtering, trusting that Claude will infer department boundaries from context

D Separate departments only by using different system prompts

CORRECT ANSWER **B**

WHY True tenant/department isolation in RAG systems requires server-side enforcement — metadata filters and permission checks applied before data is returned to the model — because the model cannot be relied upon to self-enforce access boundaries. Prompt instructions, differing system prompts, or no filtering at all are all soft controls that can be bypassed or fail silently, risking cross-department data exposure.

Q15 **An architect reviewing an agent's OAuth 2.1 integration finds that access tokens are valid for 24 hours and refresh tokens never expire or get rotated. What change best aligns with secure token-handling practice while maintaining usability?**

A Make access tokens valid indefinitely instead, to reduce refresh overhead

B **Shorten access token lifetimes appropriately for the risk level, and implement refresh token rotation with expiration/revocation so a leaked refresh token has limited value over time**

C Eliminate refresh tokens entirely and require users to re-authenticate every 24 hours

D Keep the current configuration since 24-hour access tokens are an industry best practice for all use cases

CORRECT ANSWER B

WHY Short-lived access tokens combined with rotating, expiring refresh tokens limit the damage window if a token is compromised while preserving a good user experience (no excessive re-authentication). Indefinite access tokens dramatically increase compromise risk, eliminating refresh tokens harms usability without proportional security benefit, and a flat "24-hour access token" rule ignores that appropriate lifetimes vary by risk/sensitivity of the operation.

Q16 **A security review of a remote MCP server finds that it simply forwards whatever bearer token the calling client sends along to the downstream internal API it wraps, rather than validating the token's intended audience or exchanging it for its own credential. What is this pattern called, and why is it dangerous?**

A "Token delegation," and it is safe because the downstream API performs its own signature check

B **"Token passthrough," an explicitly forbidden anti-pattern because it circumvents audience-based controls, can cause downstream logs to show requests under the wrong identity, and turns the server into a potential exfiltration proxy for tokens it never validated**

C "Token federation," a recommended best practice for simplifying multi-service authentication

D This is harmless as long as the connection uses TLS

CORRECT ANSWER **B**

WHY Token passthrough — accepting a client-supplied token and forwarding it downstream without validating that it was actually issued for this server — is a named, explicitly forbidden anti-pattern precisely because it breaks audience validation, enables confused-deputy scenarios, obscures true caller identity in downstream logs, and lets a compromised or malicious client use the server as a conduit for tokens it should never have accepted. TLS protects data in transit but says nothing about whether the token itself was validated, and this is not a legitimate delegation or federation pattern — it is the specific failure those patterns are designed to prevent.

Q17 **An architect designing OAuth scopes for a new Claude-integrated expense-reporting agent is deciding between requesting every scope the downstream finance API offers at initial setup ("read all ledgers, approve payments, modify vendor records") versus starting narrow. The team worries that requesting additional scopes later will interrupt users. What is the recommended mitigation pattern?**

A Request the broadest possible scope set up front so no future re-consent is ever needed, accepting the larger blast radius

B **Grant a minimal initial scope set, allow targeted incremental elevation only when a specific task genuinely requires it, and log every elevation event, so broad access is never assumed and every expansion leaves an audit trail**

C Request minimal scopes initially but provide no mechanism to ever elevate them, forcing users to re-authenticate from scratch for any new capability

D Skip scope design entirely and rely on the finance API's own internal business logic to silently ignore inappropriate requests

CORRECT ANSWER **B**

WHY The documented mitigation for scope-creep risk requires all three elements together: a minimal initial scope set, a targeted path for incremental elevation when genuinely needed, and logging of every elevation event — minimal scopes without an elevation path break legitimate functionality, while elevation without logging destroys the audit evidence needed to detect abuse. Requesting everything up front maximizes blast radius and makes revocation disruptive to every workflow at once, and relying on the downstream API to silently absorb over-broad requests is not a substitute for proper scope governance at the integration layer. --